Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV

Assignment No 7

Name: Soham Kulkarni
PRN: 124B1F056
Subject: Design and Analysis of Algorithms
Batch: A3
GitHub Link: https://github.com/SOHAM0007-CODER/DAA_ASSIGNMNET

1. Topic

Create a conflict-free university exam timetable using graph coloring. Each course is a vertex. Add an edge when a student takes both courses, then assign different exam slots to adjacent courses. Reduce slots where possible and study classroom-capacity constraints.

2. Course Objective

- Understand the computational complexity of algorithms.
- Select a suitable algorithm design strategy for a practical problem.

3. Course Outcome

- Model common-student conflicts and produce a valid Greedy coloring.
- Compare coloring strategies and explain how rooms affect a feasible timetable.

Theory

Graph coloring assigns a color to each vertex so that connected vertices have different colors. Here, colors represent time slots. The simple program reads course names and conflict pairs directly. It schedules courses in ID order using the first slot not already used by a scheduled conflicting course.

Greedy coloring guarantees a valid timetable for all entered conflicts, but not a minimum slot count on every graph. All shared-student conflicts must be supplied. Courses without an edge between them may use the same slot.

Working of the Algorithm

- 1. Create an undirected adjacency list from the entered conflict pairs.
- 2. For each course, mark slots already used by its scheduled neighbors.
- 3. Assign the first unmarked slot and continue to the next course.
- 4. Print each course's slot and the total number of slots used.

Complexity Analysis

Resetting the used-slot array for every course takes O(V squared) work; neighbor scans add O(E). Total time is O(V squared + E).

Time Complexity

Best Case: O(V squared + E).

Average Case: O(V squared + E).

Worst Case: O(V squared + E); course order can affect slot count.

Space Complexity

O(V + E) for adjacency lists, assigned slots and the temporary used-slot array.

Algorithm - Steps

- Step 1: Read course names and every conflicting course pair.
- Step 2: Process courses in order and assign the first legal slot.
- Step 3: Print the timetable; check that every conflict pair has different slots.

Pseudocode

```
set slot[course] = 0 for every course
for course u in ID order:
    mark every slot as unused
    for each neighbor v of u:
        if slot[v] is assigned: mark it used
    choose the smallest unused slot s
    slot[u] = s
print courses, assigned slots and total slots
```

Flow of Execution

START -> Read courses and conflicts -> Mark blocked slots -> Assign first legal slot -> Print timetable -> END.

6. Example

Courses are DAA, DBMS, OS and CN. Conflicts are (1,2), (1,3), (2,3), (3,4). The result is DAA: slot 1; DBMS: slot 2; OS: slot 3; CN: slot 1. The triangle DAA-DBMS-OS requires three slots, so this example is optimal.

Welsh-Powell colors in decreasing degree order; DSATUR chooses the course with the most distinct neighbor colors. Both are heuristics. A room extension must ensure sufficient seats and prevent double-booking; extra slots may be needed. The simple code handles conflict coloring.

Conclusion

Greedy coloring creates a valid timetable, though course order can change its slot count. DSATUR, Welsh-Powell and classroom constraints extend this basic model.